

Introduction

Every Python developer has written `del my_object` or set a variable to `None` and just trusted that the memory went away. Most never ask why. That trust is well placed — but understanding *why* it's justified is one of those things that separates someone who uses Python from someone who understands it.

This matters for more than intellectual curiosity. In interviews, "how does Python manage memory" is a classic systems-knowledge question that filters out people who've only ever worked at the application layer. In production, it explains real symptoms: a `__del__` method that doesn't fire when you expect, a service whose memory usage creeps upward despite objects being "deleted," or a cache of parent-child objects that quietly balloons in size. None of these are mysteries once you understand the two mechanisms Python actually uses to reclaim memory.

We're going to build that understanding from the ground up — no prior experience with manual memory management (C, C++, Rust) required. Python was specifically designed to hide these details from you, so we're just going to look behind the curtain.

Problem Overview

Stated as a question you might get asked in an interview or need to answer while debugging: **when you remove the last reference to a Python object, when — and how — does the memory actually get freed?**

The honest answer isn't "there's a garbage collector that cleans up periodically." That's the mental model most people bring over from Java or Go, and it's wrong for Python's *primary* mechanism. Python actually uses two distinct systems working together:

1. **Reference counting** — the primary, immediate mechanism.
2. **A generational cycle collector** — a backup mechanism that only exists to catch a specific failure mode of reference counting: reference cycles.

Understanding both, and specifically understanding *why* the second one has to exist, is the whole problem.

Example

```
import sys

class Node:
    def __init__(self, name):
        self.name = name

a = Node("A")
print(sys.getrefcount(a)) # 2 (one for 'a', one for the getrefcount arg itself)

b = a
print(sys.getrefcount(a)) # 3 (now 'a' and 'b' both point at it)

del b
print(sys.getrefcount(a)) # back to 2

del a # refcount hits 0 here → object is freed immediately
```

Nothing here calls `free()` or waits for a background sweep. The moment the reference count for the `Node` object reaches zero, CPython deallocates it, in that exact line of code.

Now the case that breaks this:

```
class Node:
    def __init__(self, name):
        self.name = name
        self.partner = None

x = Node("X")
y = Node("Y")
x.partner = y
y.partner = x # x and y now reference each other

del x
del y
# Neither object's refcount ever reaches 0 –
# each still holds one reference from the other.
```

After `del x` and `del y`, your program can no longer reach either object — but their internal refcounts are still 1, not 0, because they're holding onto each other. Reference counting alone will never free this memory. This is exactly the gap the cycle collector exists to close.

Intuition

Here's how an engineer arrives at this design without being told the answer up front.

Start with the goal: free memory as soon as it's safe to, without scanning the whole heap constantly (too slow) and without leaking (unacceptable). Reference counting is the obvious first move — attach a counter to every object, increment it when a new reference is made, decrement it when a reference goes away, and free the object the instant the counter hits zero. It's cheap, it's local (no full heap scan needed), and it's deterministic — you know exactly when the free happens.

But refcounting has exactly one structural blind spot: cycles. Any group of objects that reference *each other* but are unreachable from anywhere else will have a refcount that never drops to zero, because they're still counting each other's references. Refcounting literally cannot see this — it only tracks "how many things point at me," not "can the program actually reach me."

The fix requires a different question entirely. Instead of "how many references does this object have," ask "starting from everything my program can currently touch — global variables, local variables, stack frames — can I walk a path to this object?" That's **reachability**, and it's the same idea every mark-and-sweep or tracing garbage collector uses, in Java, Go, JavaScript, or anywhere else. The clever part of Python's design is that it doesn't replace refcounting with this — it keeps refcounting as the fast path for the 99% case, and only pays the cost of a full reachability scan for the objects that survive long enough to be suspicious.

That "survive long enough" idea is where generations come in. Newly created objects die young most of the time (temporary variables, function locals), so it makes sense to scan them frequently. Objects that survive several scans are statistically much more likely to be long-lived, so scanning them constantly would be wasted work. Python sorts objects into generation 0, 1, and 2, and checks each generation less often as objects prove they're not garbage.

Brute Force Solution

If you were designing this without any cleverness, the "brute force" approach is a **stop-the-world mark-and-sweep over the entire heap**, run either constantly or on a fixed timer.

Idea: Periodically pause the program, walk every reachable object from every root (globals, stack frames, etc.), mark what's reachable, then sweep — free everything unmarked.

Advantages: - Handles cycles correctly, since it's based on reachability, not counting. - Simple mental model — one mechanism instead of two.

Disadvantages: - No object is ever freed immediately — even a `Node()` with an obviously-zero refcount waits for the next sweep. - Every sweep must touch the *entire* heap, no matter how much of it is actually garbage. - Pause times grow with heap size, which is a real problem for latency-sensitive programs.

Time complexity: $O(n)$ per collection, where n is the number of live objects — but it must run this scan even when almost nothing changed. **Space complexity:** $O(n)$ for the mark set/visited tracking during traversal.

This is roughly what pure tracing garbage collectors (like the JVM's) do, and it's a legitimate design — it's just not what CPython chose as its *primary* mechanism, because CPython optimizes for the common case where cycles are rare.

Optimal Solution

CPython's actual design is a hybrid, and each half is doing a job the other can't:

Mechanism	Handles	When it runs	Cost
Reference counting	The 95%+ case: objects with no cycles	Immediately, inline, every time a reference changes	$O(1)$ per operation
Generational cycle collector	The remaining case: reference cycles	Periodically, triggered by allocation thresholds	$O(\text{objects in that generation})$

Step 1 — Reference counting does the heavy lifting. Every object carries a counter. Every assignment, function call, or container insertion that adds a reference increments it; every `del`, scope exit, or reassignment that removes one decrements it. Hit zero → freed immediately, same line of code, no waiting.

Step 2 — New objects go into generation 0. Nearly everything starts here, because most objects (loop variables, temporary calculation results) never outlive their function.

Step 3 — Survivors get promoted. An object that survives a generation-0 scan (i.e., it's still referenced by something, but might be part of an unreachable cycle the scanner hasn't caught yet, or is just genuinely long-lived) moves to generation 1, then eventually generation 2. Each higher generation is scanned less frequently, since long-lived objects are statistically unlikely to suddenly become garbage.

Step 4 — The cycle collector runs a reachability scan on a generation, not the whole heap. It starts from your program's roots and walks outward, marking everything reachable. Anything left unmarked afterward — including a self-referencing pair like `x / y` from the earlier example — is unreachable garbage and gets collected, regardless of what its individual refcount says.

This is why the rule to internalize is: **Python frees objects that are unreachable, not just unreferenced.** Refcounting is a fast approximation of unreachability that fails exactly in the cycle case, and the generational scanner exists solely to cover that gap.

Python Code

```
import gc
import sys

class Node:
    """A simple node that can reference another node, potentially forming a cycle."""

    def __init__(self, name):
        self.name = name
        self.partner = None

    def __del__(self):
        print(f"Node({self.name}) is being freed")

def demonstrate_refcounting():
    a = Node("A")
    print("Refcount after creation:", sys.getrefcount(a))

    b = a
    print("Refcount after second reference:", sys.getrefcount(a))

    del b
    del a # refcount hits 0 -> freed immediately, __del__ fires here

def demonstrate_cycle():
    x = Node("X")
    y = Node("Y")
    x.partner = y
    y.partner = x # cycle formed

    del x
    del y # refcounts do NOT reach 0 -- __del__ does not fire yet

    print("Before manual collection:", gc.get_count())
    collected = gc.collect() # forces the generational scanner to run
    print(f"Objects collected: {collected}")

if __name__ == "__main__":
    demonstrate_refcounting()
    demonstrate_cycle()
```

Code Walkthrough

- `sys.getrefcount(a)` reports the reference count for `a`, plus one for the temporary reference the function call itself creates — that's why a "fresh" object shows 2, not 1.
- `__del__` is defined so we can *see* exactly when CPython actually deallocates each object — it only fires the instant the object is truly freed.
- In `demonstrate_refcounting`, `del a` is the moment the refcount hits zero, so `__del__` prints immediately, in that line.
- In `demonstrate_cycle`, `x` and `y` hold references to each other via `.partner`. After `del x` and `del y`, nothing in the program can reach either object, but each still holds one reference from the other — refcounts stay above zero.
- `gc.get_count()` shows how many objects currently sit in generations 0, 1, and 2, waiting to be scanned.
- `gc.collect()` forces an immediate reachability scan. This is what finally frees the cyclic pair, and only now do their `__del__` methods fire.

Dry Run

Walking through `demonstrate_cycle()` :

1. `x = Node("X")` → `x` 's refcount = 1.
2. `y = Node("Y")` → `y` 's refcount = 1.
3. `x.partner = y` → `y` 's refcount = 2 (`y` variable + `x.partner`).
4. `y.partner = x` → `x` 's refcount = 2 (`x` variable + `y.partner`).
5. `del x` → `x` 's refcount drops to 1 (still held by `y.partner`). Not freed.
6. `del y` → `y` 's refcount drops to 1 (still held by `x.partner`). Not freed.
7. At this point, both objects exist in memory, each with refcount 1, and neither is reachable from any live variable.
8. `gc.collect()` walks from the program's roots, finds no path to either `Node`, marks both as garbage, and frees them — `__del__` fires for both, only now.

Complexity Analysis

- **Reference counting:** $O(1)$ time per reference change — increment/decrement is a single operation, independent of heap size. Space overhead is $O(1)$ per object (one integer counter).
- **Cycle collection:** $O(k)$ time per run, where k is the number of objects in the generation being scanned (not the whole heap) — this is why generations matter, they bound the scan size. Space is $O(k)$ for tracking reachability during the scan.

These are correct because refcounting never has to look beyond the single object being modified, and the generational design deliberately avoids ever scanning the full heap in the common case — only objects that have survived previous scans, and are therefore suspected of being long-lived or cyclic, get re-examined.

Alternative Solutions

Mark-and-sweep only (no refcounting), as JVM/Go use: viable, and it correctly handles cycles without a second mechanism. The tradeoff is that *nothing* is freed deterministically — even simple, cycle-free objects wait for the next collection pass, which shows up as GC pause times. Python's hybrid trades a small amount of design complexity for immediate, predictable deallocation in the common case.

Reference counting only (no cycle collector): this is what Python effectively looked like conceptually before the cycle detector was added — and it leaks memory permanently on any cyclic structure. Not viable for a general-purpose language.

Weak references (`weakref` module): not a replacement for the collector, but a complementary tool — used to deliberately break cycles you control (e.g., a parent-child relationship where the child holds a `weakref` back to the parent) so refcounting alone can free the structure without needing the cycle collector at all.

Edge Cases

- **Object with zero references from the start:** never happens in valid Python — creation always produces at least one reference.
- **Self-referencing single object** (`a.self_ref = a`): a cycle of size one — refcounting fails the same way, and the generational collector catches it the same way.
- **Long chain with no cycle** ($A \rightarrow B \rightarrow C \rightarrow D$): refcounting handles this fine; deleting the head reference cascades refcounts to zero down the chain immediately.
- **Objects that opt out of cycle detection:** container-free objects that can't participate in cycles (ints, strings, most simple values) are excluded from the cycle collector's tracking entirely — a memory/performance optimization.
- **Cycles involving `__del__`** : in older Python 2, objects in a cycle with a `__del__` method couldn't be collected at all and leaked; Python 3.4+ (PEP 442) fixed this, and the code above works because of that fix.

Common Mistakes

1. **Assuming `del` always frees memory immediately.** It only does if the refcount hits zero — cyclic objects will linger until the next collection cycle.
2. **Believing all Python memory management is "garbage collection" in the tracing-GC sense.** Reference counting is the primary mechanism; the "gc" module only handles the cycle-catching backup.
3. **Expecting `__del__` to fire deterministically for cyclic objects.** It won't fire until `gc.collect()` (automatic or manual) actually runs.
4. **Manually calling `gc.collect()` everywhere "to be safe."** It's a full scan of a generation — calling it constantly defeats the performance purpose of generations and rarely helps in practice.
5. **Confusing `sys.getrefcount()` output with the "real" count.** It's always one higher than expected because the function call itself creates a temporary reference.

Interview Questions

- Why does Python need both reference counting and a cycle collector — why not just one?
- Walk through what happens in memory when two objects reference each other and both variables are deleted.
- What are Python's garbage collection generations, and why does object age matter?
- How would you deliberately avoid creating a reference cycle in a parent-child data structure?
- What's the difference between "unreferenced" and "unreachable," and why does that distinction matter here?

Similar Problems

- **Detect Cycle in a Linked List (LeetCode 141)** — the same reachability/traversal idea underlies both: can you get back to where you started by only following pointers?
- **Clone Graph (LeetCode 133)** — requires tracking visited nodes to avoid infinite loops, conceptually mirroring how the cycle collector avoids re-walking the same objects.
- **Course Schedule / Cycle Detection in a Directed Graph (LeetCode 207)** — same underlying graph-cycle-detection logic, applied to a scheduling problem instead of memory.

Key Takeaways

- Python frees most objects the instant their reference count hits zero — this is fast, deterministic, and happens inline.
- Reference counting has one blind spot: objects that reference each other in a cycle never reach a refcount of zero on their own.
- The generational cycle collector exists solely to catch that blind spot, by asking "is this reachable from something alive," not "does something point at this."
- Generations (0, 1, 2) exist purely as a performance optimization — young objects are checked often, long-lived ones rarely.
- The underlying idea — reachability over reference counting — is how every major garbage-collected language ultimately decides what's garbage.

Watch the Video

For the live walkthrough — watching `sys.getrefcount` change in real time and seeing `gc.collect()` free a stuck cycle on screen — check out the full video here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series that takes tools developers use every single day and actually opens the hood on how they work. No prerequisite background required — just curiosity about what's happening underneath the abstractions you rely on.

Call To Action

If this made Python's memory model click for you, drop a like on the video — it's the clearest signal that this format is working. Subscribe on YouTube for more of these deep dives, and subscribe to the Substack for the written companion pieces that go even deeper than the video. Got a topic you want opened up next? Leave a comment — and if you know another developer who'd get value from this, share it with them.

Quick Review

Refcounting: what triggers CPython to free an object?

When an object's refcount hits zero — no more names or containers point to it — CPython frees it immediately.

Cost of refcount inc/dec on assignment?

O(1) per assignment; CPython just bumps an integer field on the object header.

Extra space cost refcounting adds per object?

One machine word (ob_refcnt) stored in every object's header, always.

Common bug: two objects referencing each other never freed?

Reference cycle — refcount never reaches zero even when unreachable from your code; refcounting alone leaks.

Refcounting vs generational cycle collector — when does each run?

Refcounting frees instantly on every dereference; the gen0/1/2 cycle collector runs periodically to catch cycles refcounting misses.

Interview: does gc always find every unreachable cycle?

No — objects with `__del__`-like finalizers or C extension objects that don't support tracking can escape detection.